

Difference between JPA, Hibernate and Spring Data JPA

A walkthrough of how these three layers relate to each other — a specification (JPA), an implementation of that specification (Hibernate), and an abstraction built on top of a JPA provider (Spring Data JPA) — with a side-by-side code comparison.

1. What Each One Actually Is

Java Persistence API (JPA)

- JPA is a **specification** — formally, **JSR 338** — that defines a standard set of rules for persisting, reading, and managing data between Java objects and a relational database.
- It **does not contain any concrete implementation**. JPA is just interfaces and annotations (`@Entity`, `@Id`, `EntityManager`, etc.) describing *what* should happen, not *how*.
- **Hibernate is one implementation** of this specification — other providers exist too (EclipseLink, OpenJPA), and any of them can be swapped in as long as they honor the JPA contract.

Hibernate

- Hibernate is an **ORM (Object-Relational Mapping) tool** that **implements the JPA specification**.
- It's the engine that actually does the work: generating SQL, managing the Session/EntityManager, tracking object state, and talking to the JDBC driver.
- Because it implements JPA, code written against JPA's standard interfaces (`EntityManager`, `@Entity`, JPQL) will run on Hibernate — but Hibernate also offers extra, non-standard features beyond what JPA requires (e.g. its own native **Session** API, additional annotations, and caching options).

Spring Data JPA

- Spring Data JPA **does not implement JPA itself** — it has no ORM engine of its own.
- Instead, it sits **one level higher**: it's an abstraction layer over whatever JPA implementation provider you're using underneath (typically Hibernate).
- Its main job is to **reduce boilerplate code** — instead of writing DAO classes with manual Session/EntityManager handling, you declare a repository interface and Spring Data generates the implementation for you at runtime.
- It also **manages transactions** declaratively (via `@Transactional`), so you don't need to manually call begin/commit/rollback in your service code.

How the Three Layers Stack Up

Layer	Role
Spring Data JPA	Highest-level abstraction — repository interfaces, less boilerplate, declarative transactions
JPA	Specification / contract in the middle — defines the standard API (EntityManager, @Entity, JPQL)
Hibernate	Concrete implementation underneath — generates SQL and talks to JDBC

Reading top to bottom: your application code calls **Spring Data JPA** repositories → which delegate to the **JPA** standard API (EntityManager) → which is backed by **Hibernate** as the actual JPA provider doing the SQL generation and database work.

2. Code Comparison — Hibernate vs. Spring Data JPA

The clearest way to see the difference in practice is to compare how the exact same operation — "create an Employee record" — is written in each style.

Hibernate (native API)

Using Hibernate directly, the developer manages the **Session** and **Transaction** lifecycle by hand — open the session, begin a transaction, save, commit, catch and roll back on failure, and always close the session.

```
/* Method to CREATE an employee in the database */
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;

    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    } catch (HibernateException e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeID;
}
```

Spring Data JPA

With Spring Data JPA, none of that plumbing is written by hand. A repository interface is declared once, and Spring generates the implementation (including the save logic) at runtime.

```
// EmployeeRepository.java
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {
}
```

```
// EmployeeService.java
@Autowired
private EmployeeRepository employeeRepository;
```

```
@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
```

What Changed, and Why

- **No manual Session/Transaction handling.** Opening a session, beginning a transaction, committing, rolling back, and closing the session are all gone from application code — Spring Data and the surrounding Spring container handle it.
- **@Transactional replaces try/commit/rollback/finally.** Declaring the method transactional tells Spring to wrap it in a transaction automatically and roll back on an unchecked exception — this is what "manages transactions" means for Spring Data JPA.
- **JpaRepository<Employee, Integer> already provides save().** Extending this interface gives basic CRUD (save, findById, findAll, delete, etc.) with zero implementation code — this is the "reduces boilerplate" benefit.
- **Hibernate hasn't disappeared** — it's still doing the actual ORM work underneath; Spring Data JPA is simply the layer generating the repository implementation and coordinating the transaction around whichever JPA provider (Hibernate, by default in most Spring Boot setups) is configured.

3. Summary Comparison

Aspect	JPA	Hibernate	Spring Data JPA
What it is	A specification (JSR 338)	An ORM implementation of JPA	An abstraction over a JPA provider
Provides an implementation	No	Yes	No — delegates to Hibernate/EclipseLink/etc.
Reduces boilerplate?	No — defines the API only	Some — vs. plain JDBC	Yes — repositories generated at runtime
Transaction handling	Standard API (EntityManager)	Manual (beginTransaction/commit/rollback)	Declarative (@Transactional)
Typical usage	Programming against the standard JPA API	Session API or as the JPA engine underneath	Repository interfaces in Spring apps

Reference: DZone — "What Is the Difference Between Hibernate and Spring Data JPA"

(dzone.com/articles/what-is-the-difference-between-hibernate-and-spring-1); JavaWorld — "What is JPA? Introduction to the Java Persistence API" (javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html)